SMT over finite fields, ring-theoretically

companion note to *Training for Constraining*, Devcon 8

1. Constraints are an ideal; soundness is a question about rational points

A circuit over $\mathbb{F}_p$ is a list of polynomials $f_1, \ldots, f_m \in R = \mathbb{F}_p[x_1, \ldots, x_n]$ (public and private wires alike). The accepted witnesses are exactly the $\mathbb{F}_p$-rational points of $\mathcal{I} = \langle f_1, \ldots, f_m \rangle$, i.e. $V_{\mathbb{F}_p}(\mathcal{I}) = V(\mathcal{I}) \cap \mathbb{F}_p^n$. Soundness says every accepted witness satisfies the specification R. The solver refutes its negation:

$$\exists\, (w, x) \in \mathbb{F}_p^n : \quad f_1 = \cdots = f_m = 0 \ \wedge\ \neg R(w, x),$$

so it needs $\neg R$ as polynomials too. A disequation $g \neq 0$ becomes an equation by **Rabinowitsch's trick**: adjoin a fresh variable t and assert $t\,g - 1 = 0$. Over a field this is equivalent to $g \neq 0$; algebraically it is the localisation $R[g^{-1}]$. Hence the whole query is one ideal $J = \langle f_1, \ldots, f_m,\ t_1 g_1 - 1, \ldots, t_k g_k - 1 \rangle$, and the question is: *does $V(J)$ contain a point with all coordinates in $\mathbb{F}_p$?*

2. The dictionary

Logic / solver	Commutative algebra
conjunction of equations	ideal generated by the polynomials (sum of ideals)
disequation $g \neq 0$	$t\,g - 1$: g becomes a unit (localisation)
UNSAT over the algebraic closure	$1 \in J$ (weak Nullstellensatz: $V(J) = \varnothing \iff 1 \in J$)
UNSAT over $\mathbb{F}_p$ exactly	$1 \in J + \langle x_i^p - x_i \rangle$ (field equations cut V down to $\mathbb{F}_p^n$)
SAT with model	an $\mathbb{F}_p$-point of $V(J)$; the model is its coordinates
unsat core	which generators occur in the certificate $1 = \sum h_i f_i$

3. What cvc5 actually computes (no sampling anywhere)

Refutation. It computes a Gröbner basis of J (Buchberger/F4) for a monomial order and reduces 1. The reduced basis is $\{1\}$ iff $1 \in J$, and the computation produces an explicit **Nullstellensatz certificate** $1 = \sum_i h_i f_i$: a finite identity that is a proof of emptiness for all p^n candidate points at once. If the h_i have integer coefficients the identity holds in every commutative ring, which is why the same UNSAT is returned over $\mathbb{F}_{11}$ and over the 254-bit BN254 field in the same few milliseconds. **Model finding.** If the basis is not $\{1\}$, $V(J) \neq \varnothing$ over $\overline{\mathbb{F}}_p$ but an $\mathbb{F}_p$-point still has to be found. Zero-dimensional case: a lexicographic basis is triangular, so solve the univariate element by exact root finding in $\mathbb{F}_p$ (gcd with $x^p - x$, Cantor–Zassenhaus) and back-substitute. Positive-dimensional case: fix a free variable, recompute, recurse, backtrack. Each step is exact elimination; the returned model is re-checked against every constraint. **The gap.** Completeness over $\mathbb{F}_p$ needs the field equations $x_i^p - x_i$, of degree $p \approx 2^{254}$: hopeless, so cvc5 omits them. Consequences: UNSAT is a proof (even over $\overline{\mathbb{F}}_p$); SAT comes with a checkable witness; and a third answer, UNKNOWN, exists when the point search is exhausted or times out. A fuzzer can only ever say SAT; it has no object corresponding to the certificate.

4. Worked example: the selector

Constraints $f_1 = z - sx - (1 - s)y$ and $f_2 = s(s - 1)$; negated spec $g_1 = t_1(z - x) - 1$, $g_2 = t_2(z - y) - 1$. Reducing modulo f_1,

$$(z - x)(z - y) \ \equiv\ (1 - s)(y - x) \cdot s(x - y) \ =\ s(s - 1)(x - y)^2 \pmod{f_1},$$

and f_2 kills the right-hand side. Writing $h = z + sx + (1 - s)y - x - y$, the identity (verified by expansion)

$$\boxed{1 \ =\ t_1 t_2 \big[h\, f_1 + (x - y)^2 f_2\big] \ -\ \big(g_1 g_2 + g_1 + g_2\big)}$$

is the certificate: $1 \in J$, so the repaired selector is sound in every field. Delete f_2 and $J' = \langle f_1, g_1, g_2 \rangle$ has the $\mathbb{F}_{11}$-point $(x, y, z, s) = (3, 7, 5, 6)$ with $t_1 = 2^{-1} = 6$, $t_2 = (-2)^{-1} = 5$: cvc5 answers SAT and prints it. The Lean theorem `selector_sound` is the same argument in two cases via `mul_eq_zero`. **Where the field axiom enters.** Over $\mathbb{Z}/6$, $s = 3$ satisfies $s(s-1) = 0$ and gives $z = 3x + 4y \notin \{x, y\}$ for $(x, y) = (0, 1)$, yet the certificate is a valid ring identity. No contradiction: Rabinowitsch encodes "$g \neq 0$" as "g is a unit", and 4 is not a unit mod 6. The two notions coincide exactly in a field; that coincidence is what "sound over every field" really rests on.

5. cvc5 versus Zirgen: different layers of the stack

cvc5 is a general SMT solver (Stanford/Iowa). Its finite-field theory `QF_FF` (Ozdemir, Kremer, Tinelli, Barrett, CAV 2023; split Gröbner bases, CAV 2024) decides polynomial systems over $\mathbb{F}_p$ as in §3. Input: constraints in SMT-LIB. Output: SAT+model, UNSAT, or UNKNOWN. It *analyses* circuits and is the backend of Picus; it never produces one.

Zirgen is RISC Zero's circuit DSL and MLIR-based compiler. It *produces* circuits: AIR constraints plus witness generation, emitted as Rust/C++/CUDA/Metal prover and verifier code for the RISC Zero zkVM. It decides nothing. Verification reaches it from outside: Nethermind's Lean model of Zirgen's MLIR with a verification-condition generator, and Veridise's zirgen-to-LLZK frontend, through which Picus (hence cvc5) and zkLean become available.

In the language of this note: Zirgen is where the ideal J gets written down; cvc5 is one of the procedures that decides whether J has a rational point. Compiler versus decision procedure, not competitors.

References. Cox–Little–O'Shea, *Ideals, Varieties, and Algorithms* (Nullstellensatz, Gröbner bases). Ozdemir et al., *Satisfiability Modulo Finite Fields*, CAV 2023 (ePrint 2023/091). Ozdemir et al., *Split Gröbner Bases for Satisfiability Modulo Finite Fields*, CAV 2024 (ePrint 2024/572). Pailoor et al., *Automated Detection of Under-Constrained Circuits in ZK Proofs*, PLDI 2023 (Picus). `https://github.com/risc0/zirgen`; `https://github.com/NethermindEth/risczero-fv`; `https://github.com/project-llzk/llzk-lib`.

Why split Gröbner bases, where they stop, and when to switch to Lean

6. The bitsum wall

Take the simplest range check there is, for $p \geq 2^b$:

$$\bigwedge_{i=1}^{b} X_i(1 - X_i) = 0, \qquad X_1 + 2X_2 + \cdots + 2^{b-1}X_b = 0, \qquad X_b Z = 1.$$

A human refutes it in one line: the bits are Boolean, so the bitsum forces every $X_i = 0$, contradicting $X_b Z = 1$. A full Gröbner basis for this system takes time *exponential in* b. The 2023 cvc5 solver could not decide determinism of circomlib's `Num2Bits` at $b = 32$ "even in a week". The diagnosis (their §3.2) is sharp: the bitsum alone is already a Gröbner basis, the bit constraints alone are already a Gröbner basis, and it is *their combination* that blows up. Translating to bit-vectors is a dead end because the surrounding equations are genuinely field equations. And the pattern is not exotic: 98% of public Circom projects use bitsums, and 64% of the variables in circomlib's SHA-2 sit inside one. This is slide 24 of the workshop at industrial scale.

7. The fix: several small bases plus one non-algebraic fact

A **split Gröbner basis** for I is a tuple $(B_1, \ldots, B_k)$ of Gröbner bases with $\langle B_1 \cup \cdots \cup B_k \rangle = I$. Each B_i answers ideal membership for its own subsystem only; in exchange it may be cheap. The procedure Split refines the tuple by passing polynomials between bases, and a hook extraProp may inject any polynomial provably in I by *other* means. The concrete instance BitSplit uses $k = 2$: a *sparse* basis holding everything except bitsums, and a *linear* basis holding all linear polynomials including bitsums, so that a bitsum and its bit constraints never meet in one Buchberger run. Its single propagation rule is **congruence of binary representation**: if $Y - \sum 2^{i-1}X_i$ and $Y' - \sum 2^{i-1}X'_i$ are present, all bits are known Boolean, $m < \log_2 |\mathbb{F}|$, and $Y - Y' \in I$, then every $X_i - X'_i \in I$. For `Num2Bits` determinism that is the whole proof: with $f = X'_j - X_j$ propagated and $g = (X'_j - X_j)Z - 1$ the disequation, $1 = fZ - g \in I$. Soundness and termination are proved (their Thm. 3), and the search phase is complete in principle. Numbers at a 300 s limit over 1140 benchmarks:

	all solved	`QED2` (100)	CirC determinism (640)	CirC soundness (100)
Monolithic (2023)	475	13	191	38
BitSplit (2024)	969	100	582	59
BitSplit without the bitsum propagator	344	0	131	34

8. What it does not fix

1. **The worst case is unchanged.** "In the worst case, BitSplit builds a GB for the full system"; complexity stays doubly exponential. The gain is heuristic, and the third row above shows it is almost entirely the propagator.
2. **The propagator matches a syntactic pattern.** It fires only on $Y - \sum 2^{i-1}X_i$ with $m < \log_2 |\mathbb{F}|$ and bits already known Boolean. Signed-digit or base-2^k limb decompositions, bitsums rewritten by earlier algebra, and any decomposition with $2^m > p$ (the slide-24 pitfall, where representation is not unique) receive no help and fall back to raw Gröbner computation.
3. **Integer semantics are out of reach.** The authors' own account of what remains: "BitSplit struggles only with division and remainder; verifying their determinism would require understanding that integer division is deterministic, as encoded in field constraints." That is precisely the quotient-and-range argument of slides 25–26 and of foreign-field gadgets like fake GLV.
4. **Determinism is not soundness.** The breakthrough is on determinism queries, $\Phi(X, Y) \wedge \Phi(X, Y') \wedge Y \neq Y'$, whose specification writes itself. Queries against a real specification R (CirC soundness, 59/100) remain hard, and the selector of §4 is sound for a reason determinism alone cannot express.
5. **One query per bitwidth and per field.** Each benchmark fixes b and p. Nothing is learned about $b + 1$.
6. **Incomplete internal propagation and thin CDCL(T) integration**, both acknowledged: linear-to-sparse propagation is incomplete (their Example 1, and completing it did not help), and the solver "constructs no theory lemmas".

9. When to rely on a Lean proof instead

A Lean proof is not a search, so it has no unknown: a human supplies the argument and the kernel checks it, and the cost is human time rather than solver time. That cost is worth paying exactly where the list above says the solver stops. **Parametric statements.** "Binary representation is unique whenever $2^b \leq p$" is one theorem for all b and all p, a short induction; the solver needs one query per (b, p) and times out past $b \approx 32$ for the hard operators. **Integer semantics.** The argument "$xy - z - 3k \in \{-5, \ldots, 4\}$ and the only multiple of 11 there is 0" is a lift from $\mathbb{F}_p$ to $\mathbb{Z}$ followed by linear arithmetic; in Lean that is `omega` after `ZMod.val` bookkeeping, and it scales to 254-bit moduli and multi-limb representations unchanged. This is the division-and-remainder gap, closed. **Specifications, not just determinism.** `selector_sound` and `scalar_cancellation` in the repository state $C \Rightarrow R$ against the intended relation, over an arbitrary field and an arbitrary module, which no determinism query can. **Group-theoretic hypotheses.** Prime-order subgroup membership and $v \not\equiv 0 \pmod{\ell}$ are hypotheses about a module over $\mathbb{F}_\ell$, not polynomial identities in $\mathbb{F}_r$; they are natural in Lean and have no SMT encoding of practical size.

Working rule. Run the push-button tool first: sat is an exploit, unsat closes a fixed-size gadget. Escalate to Lean when the answer is unknown, when the property is parametric in b or p, when the proof needs integers or groups rather than field identities, or when the property is soundness against a specification rather than determinism. The caveats of page 1 still apply: the theorem is about the model, so the circuit-to-model step (LLZK, zkLean) is where the remaining trust sits.